

Authentication - JWT token

This page contains details on JWT provided by AFTER

- [What is JSON Web Token?](#)
- [What is the JSON Web Token structure?](#)
 - Register claims
 - Public claims
 - Specific claims
 - Not used claims
- [How create a JSON Web Token ?](#)
- [How retrieve AFTER Access Token ?](#)
- [How manage user attributes on Keycloak ?](#)

What is JSON Web Token?

JSON Web Token (JWT) is an open standard ([RFC 7519](#)) that defines a compact and self-contained way for securely transmitting information between parties as a JSON object. This information can be verified and trusted because it is digitally signed. JWTs can be signed using a secret (with the **HMAC** algorithm) or a public/private key pair using **RSA** or **ECDSA**.

On AFTER and mainly on others services deployed in Airbus, JWT is used to manage the authentication and authorization of a HTTP requests. Once the user is logged in, each subsequent request will include the JWT, allowing the user to access routes, services, and resources that are permitted with that token. Single Sign On is a feature that widely uses JWT nowadays, because of its small overhead and its ability to be easily used across different domains.

What is the JSON Web Token structure?

In its compact form, JSON Web Tokens consist of three parts separated by dots (.), which are:

- Header
- Payload
- Signature

Header	The header <i>typically</i> consists of two parts: the type of the token, which is JWT, and the signing algorithm being used, such as HMAC SHA256 or RSA. For example: <pre>{ "alg": "HS256", "typ": "JWT" }</pre> Then, this JSON is Base64Url encoded to form the first part of the JWT
Payload	The second part of the token is the payload, which contains the claims. Claims are statements about the user and additional data. There are different types of claims: • Registered claims: These are a set of predefined claims which are not mandatory but recommended, to provide a set of useful, interoperable claims. (See details) • Public claims: These can be defined as we want, but to avoid collisions they should be defined in the IANA JSON Web Token Registry (See details) • Specific claims: These are a set of specific claims designed by AFTER and used to structure information and roles not covered by the public claims. (See details) The payload is then Base64Url encoded to form the second part of the JSON Web Token.
Signature	To create the signature part you have to take the encoded header, the encoded payload, a secret, the algorithm specified in the header, and sign that. The signature is used to verify the message wasn't changed along the way, and, in the case of tokens signed with a private key, it can also verify that the sender of the JWT is who it says it is.

The output is three Base64-URL strings separated by dots that can be easily passed in HTML and HTTP environments, while being more compact when compared to XML-based standards such as SAML.

The following shows a JWT that has the previous header and payload encoded, and it is signed with a secret.

HTTP Header	Value
AUTHORIZATION	Bearer eyJhbGciOiJIUzI... fFxQ_eVTpzMz6qQ_d3iXwqj3HGLOR7sI

Whenever the user wants to access a protected route or resource, the user agent should send the JWT, typically in the **Authorization** header using the **Bearer** schema. The content of the header should look like the following:

```
Authorization: Bearer <token>
```

Here is an example of an access token payload provided by AFTER:

```
{
  "exp": 1660230875,
  "iat": 1660230575,
  "jti": "5ded905b-874e-4a0e-b91a-0447faf3e499",
  "iss": "https://keycloak.after-dev.eu.airbus.corp/auth/realms/fti-mobile-after",
  "aud": [
    "test-vault",
    "account",
    "user-portal-cli"
  ],
  "sub": "761154af-9576-43b8-a266-4aa9a1d9b0dd",
  "typ": "Bearer",
  "azp": "mobilesso-cli",
  "session_state": "b734076a-31b2-4ac1-b7e5-e04b037686ce",
  "acr": "1",
  "scope": "openid profile after-abac email",
  "sid": "b734076a-31b2-4ac1-b7e5-e04b037686ce",
  "company_type": "SUBCONTRACTOR",
  "access_provisionner": [
    "AH_FT",
    "FOA"
  ],
  "roles": [
    "APP_ADMIN_FIDES",
    "APP_ADMIN_AFTER",
    "offline_access",
    "default-roles-fti-mobile-after",
    "JOB_FLIGHT_TESTS",
    "uma_authorization"
  ],
  "access_data": [
    "FOMAX",
    "AH_HUMS",
    "FOA"
  ],
  "preferred_username": "ng496a2",
  "given_name": "Gerald",
  "upn": "gerald.g.jacobson.external@airbus.com",
  "service": "IVID",
  "name": "Gerald JACOBSON",
  "company": "SOPRA STERIA GROUP SA",
  "attributes": {
    "helicopter": {
      "hums": true,
      "ft": false
    },
    "aircraft": {
      "fomax": true
    }
  },
  "company_code": "EXT",
  "department": "Subcontractor for IVID",
  "family_name": "JACOBSON",
  "email": "gerald.g.jacobson.external@airbus.com"
}
```

Register claims

None of the claims defined below are intended to be mandatory to use or implement in all cases, but rather they provide a starting point for a set of useful, interoperable claims.

Claim attribute	Definition
-----------------	------------

iss	The "iss" (issuer) claim identifies the principal that issued the JWT. In the case of AFTER, all access token contains the following value: <pre>{ "iss": "http://keycloak-http.keycloak.svc.cluster.local/auth/realms/fti-mobile-after", ... }</pre>
sub	The "sub" (subject) claim identifies the principal that is the subject of the JWT. On AFTER, the access token contains the technical ID of the user provided by keycloak: <pre>{ "sub": "8c78b5d6-add2-44ee-84f0-bdc723d8d32d", ... }</pre>
aud	The "aud" (audience) claim identifies the recipients that the JWT is intended for. On AFTER, the access token contains the technical ID of the user provided by keycloak: <pre>{ "aud": "account", ... }</pre>
exp	The "exp" (expiration time) claim identifies the expiration time on or after which the JWT MUST NOT be accepted for processing. The processing of the "exp" claim requires that the current date/time MUST be before the expiration date/time listed in the "exp" claim. Here is a web site which translates number in human reading date : https://www.epochconverter.com/

Public claims

Here is the list of public claims provided by the access token of AFTER:

Claim attribute	Definition
iat	The "iat" (issued at) claim identifies the time at which the JWT was issued. This claim can be used to determine the age of the JWT. Its value MUST be a number containing a NumericDate value. <i>Example: 1631002723</i>
jti	The "jti" (JWT ID) claim provides a unique identifier for the JWT. <i>Example: 6f13abb0-0940-4ce0-b364-c3ee9fd5aac9</i>
typ	The "typ" (type) Header Parameter defined by is used by JWT applications to declare the media type of this complete JWT. This is intended for use by the JWT application when values that are not JWTs could also be present in an application data structure that can contain a JWT object; the application can use this value to disambiguate among the different kinds of objects that might be present. On AFTER, the access token contains the following line: <pre>{ "typ": "Bearer", ... }</pre>
azp	Authorized party - the party to which the ID Token was issued On AFTER, the access token contains the following line: <pre></pre>

acr	String specifying an Authentication Context Class Reference value that identifies the level of authentication, could be "0" or "1". The value "0" indicates the End-User authentication did not meet the requirements of ISO/IEC 29115 [ISO29115] level 1. Authentication using a long-lived browser cookie, for instance, is one example where the use of "level 0" is appropriate. Authentications with level 0 SHOULD NOT be used to authorize access to any secured resource. On AFTER, the Authentication Context Class Reference will be always "1" : <pre>{ "acr": "1", ... }</pre>
scope	The value of the scope claim is a JSON string containing a space-separated list of scopes associated with the token. On AFTER, the access token contains the following line: <pre>{"scope": "email profile", ... }</pre>
name	Full name of the user <i>Example: Jean Dupont</i>
preferred_username	Shorthand name by which the End-User wishes to be referred to. In the context of AFTER, this field contains the user ID. <i>Example: ng4587</i>
given_name	Contain the first name of the user. <i>Example: Jean</i>
family_name	Contains the last name of the user. <i>Example: dupont</i>
email	Contains the email of the user. <i>Example: jean.dupont@airbus.com</i>
email_verified	True if the e-mail address has been verified; otherwise false. On AFTER, there is no process to validate the email of user, it returns the value provided by the mobileSSO. <pre>{"email_verified": false, ... }</pre>

Specific claims

Here is the list of specific claims provided by the access token of AFTER:

Claim attribute	Definition
company	Contains the company name of the user. <i>Example: "AIRBUS OPERATIONS SAS" or "SOPRA STERIA GROUP SA"</i>
company_type	This value is extracted from the claim "company" and could take the following value: ENTITY, PARTNER, ACADEMIC, SUBCONTRACTOR, AIRLINE. The mapping between companies and types is available in this document.
company_code	This value is extracted from the claim "company" and provide a simple way to identify a company. The mapping between companies and codes is available in this document.
department	Contain the Airbus department for which the user working for. <i>Example: "1VMD" or "Subcontractor for 1VMD"</i>

service	It is an extraction of the department claim to provide a simple value of the department without prefix. For example "Subcontractor for 1VMD" becomes "1VMD".
service_code	Contain a abstraction of the service value to minimize impact of internal re-organization on applications. (TODO: not yet available)
access_data	This claim helps filtering access by business domain. Here is some example: AH_FT, FOA. All these domains are defined in Keycloak. (TODO: where is the list ?)
roles	This claim contains the Keycloak roles of the user, which are a list of administration rights (Example: APP_ADMIN_FIDES) or a list of job functions (Example:"JOB_FLIGHT_TESTS"). The list of job is deducted from the siglum of the user (Example: "IV" => "JOB_FLIGHT_TESTS") (TODO: where is the list ?)

Not used claims

Here is the list of claims which are available in the access token of AFTER, but not useful in the context of a Swift application:

Claim attribute	Definition
access_provisioner	This claim is only used by the application AFTER PORTAL to attribute rights to others users. This claim is really specific and will ne be used by Swift applications
session_state	The session_state is a claim generated and managed by Keycloak which is not used. It will be probably removed in a future to reduce the size of the token.
realm_access	It is use by Keycloak to manage the keycloak rights, but not used in AFTER and Swift process. It will be probably removed in a future to reduce the size of the token.
resource_access	It is use by Keycloak to manage the keycloak rights, but not used in AFTER and Swift process. It will be probably removed in a future to reduce the size of the token.

How create a JSON Web Token ?

The creation of JWT available on Airbus network or AFTER can't not be done without access to an Airbus private key, so for obvious security reason as developer you can't create a valid one on your local computer.

But for development purpose, you can create one which could read and use by Swift and test your security configuration.

To do that, the simple is to use features provided by the web site jwt.io which provides a nice interface. For details bout its usage, have a look to the page [Security - development mode](#).

How retrieve AFTER Access Token ?

When a HTTP request is sent on AFTER, the istio gateway intercept the request and proceed to the authentication process. A the end of this process, a set of cookie are created and push on the HTTP response. As end user, you can read these cookies from your browser.

For example, on chrome open the "developer tools" (ctrl+shift+i), click on the "Application" tabsheet et open the cookies set by <https://gateway.after.eu.airbus.corp/user-portal-ui/home>. You must see 5 cookies:

Cookie	Definition
AFTER_ACCESS_TOKEN	This cookie contains the JSON Web token containing all Keycloak attributes, and send later by Swift to retrieve the OPA roles. (Expiration time: 5min)
AFTER_REFRESH_TOKEN	This one is used to refresh the AFTER_ACCESS_TOKEN without following all SSO processes. (Expiration time: 30 min)
MOBILE_ID_TOKEN	Token used to identify the user. No need to refresh it too often, but the value is defined by Airbus standard. (Expiration time: 30 min)
MOBILE_ACCESS_TOKEN	This one is used to refresh the MOBILE_ID_TOKEN. (Expiration time: 30 min)
AFTER_SSO_ORIGIN	Use to register the original url, because with potential SSO redirection this information is lost during the process and it is easier to retrieve it with a cookie.

Here is a video showing the steps: [Swift-readJWT-fromCookies.mp4](#).

How manage user attributes on Keycloak ?

The management of user configuration in Keycloak is supported by AFTER team, and only them. So if you want to customize or add [ABAC attributes](#), you need to contact them.

To know your own Keycloak configuration, you can go on the [AFTER user portal](#) and see your account details ([video](#))